

ComicCraft – AI Comic Story Creator

Build a complete Python-based web application from scratch in Visual Studio Code. ComicCraft should generate personalized comic stories, illustrations, previews, and PDFs from user-provided prompts.

1. Technology Stack

- Python 3.11+
- FastAPI + Uvicorn
- Google Gemini API for story generation
- Hugging Face Diffusers / Stable Diffusion for images
- Pillow for image processing
- ReportLab for PDF creation
- HTML, CSS, and JavaScript for the frontend
- python-dotenv for environment configuration

2. Main Features

- User inputs: story prompt, character name/description, setting, tone, art style, panel count, and additional instructions.
- Generate a structured panel-by-panel story with narration, dialogue, scene descriptions, and image prompts.
- Generate consistent comic illustrations for each panel.
- Display the finished comic in a responsive web interface.
- Regenerate an individual panel.
- Download the complete comic as a PDF.
- Provide FastAPI API endpoints and Swagger documentation.
- Include a development/mock mode so the project can run without a powerful GPU.

3. Project Structure

- ComicCraft/
 - app/main.py
 - app/config.py
 - app/schemas.py
 - app/services/gemini_service.py
 - app/services/image_service.py
 - app/services/pdf_service.py
 - app/utils/file_utils.py
 - app/static/index.html

- app/static/css/style.css
- app/static/js/app.js
- generated/images/
- generated/pdfs/
- .env.example
- .gitignore
- requirements.txt
- run.py
- README.md

4. API Endpoints

- GET / – display the ComicCraft website.
- GET /api/health – health check.
- POST /api/generate-story – generate the story using Gemini.
- POST /api/create-comic – generate story, images, and PDF.
- POST /api/regenerate-panel – regenerate one selected panel.
- GET /api/comic/{comic_id} – retrieve comic information.

5. Gemini Requirements

- Generate original stories with consistent characters and logical panel continuity.
- Return structured JSON containing title, summary, character profile, panels, narration, dialogue, scene descriptions, and image prompts.
- Validate Gemini output using Pydantic and handle API errors, invalid JSON, rate limits, and network failures.

6. Image and PDF Requirements

- Generate panel images using Diffusers and save them in generated/images/.
- Include character appearance, action, environment, lighting, camera angle, mood, and art style in image prompts.
- Use ReportLab to create a PDF containing the title, summary, panel images, narration, dialogue, and page numbers.

7. Frontend Requirements

- Create a clean comic-book-style responsive interface.
- Include a creation form, Generate button, loading status, comic preview, panel cards, Regenerate Panel button, Download PDF button, and New Comic button.
- Use JavaScript Fetch API to communicate with FastAPI without unnecessary page reloads.

8. Configuration and Security

- Store API keys only in .env; never hard-code secrets.
- Validate inputs and panel count (1–12).
- Use safe filenames and file paths.
- Do not expose API keys, stack traces, or arbitrary local files.

9. Required Files and Documentation

- Create a complete requirements.txt, .env.example, .gitignore, run.py, and README.md.
- README must explain installation, virtual environment setup, API-key configuration, running the application, image-generation requirements, and troubleshooting.
- All files must contain complete working code. Do not use pseudo-code, omitted sections, or placeholders.

10. Run Instructions

- `python -m venv .venv`
- `.venv\Scripts\activate`
- `pip install -r requirements.txt`
- Copy .env.example to .env and add the Gemini API key.
- `python run.py`
- Open `http://127.0.0.1:8000`

Final Instruction to the Coding Agent

Build and verify the entire ComicCraft project from an empty VS Code folder. Ensure all imports, API routes, frontend requests, Gemini integration, image generation, PDF generation, file paths, environment variables, and development mode work together. Return the complete source code for every required file so it can be copied directly into VS Code.